

Architecture of MobileNet

+

Aashish Mahato

Agenda

- + Fine-Tuning
- + MobileNet
- + MobileNetV1
- + MobileNetV2

The background features a light beige color with abstract, flowing dashed blue lines. There are two white circles: one in the top-left corner and another in the bottom-right corner. A solid red line curves along the bottom right edge of the frame.

Fine-Tuning

+

What is Fine-Tuning?

- + Fine-tuning is the process of taking an already trained or pre-trained machine learning model and further training it on a smaller, targeted data set so it can perform specialized tasks.
- + Fine-tuning enables developers to save time and money by building on existing models instead of creating new ones.
- + For example, a pre-trained LLM model might have a general grasp of English, but a law firm may fine-tune it to understand legal jargon specifically.

Types of Fine-Tuning

Feature extraction fine-tuning

- + Feature extraction fine-tuning, sometimes referred to as parameter-efficient fine-tuning, “freezes” the top layers of the model while the final layers train on the specified data set. This keeps most of the existing model intact, making it more computationally efficient.

Types of Fine-Tuning

Full model fine-tuning

- + Full model fine-tuning is similar to pre-training in that the entire model trains on a data set and updates the weights accordingly. This process updates the whole model, potentially leading to stability issues.
- + This fine-tuning method is more computationally intensive than feature extraction fine-tuning. It's best suited to situations where the new data set is both sizable and distinct from the original training data set.

Fine-Tuning Process

+ Fine-tuning adapts a pre-trained model to a specific task through a systematic process, as outlined below:

1. Start with a pre-trained model: MobileNetV2
2. Prepare a task-specific dataset: New Plant Diseases Dataset
3. Fine-tune the model: Use the New Plant Diseases Dataset to fine-tune the MobileNetV2 model.
 - Feature Extraction Fine-tuning (trained for 10 epochs)
 - Partial Fine-tuning (i.e. unfreeze the final 10 layers and trained for 5 epochs)

Fine-Tuning Process

4. Validate the model: Test the fine-tuned MobileNetV2 model on a validation set to measure its performance and ability to generalize to new data.
5. Prepare the fine-tuned MobileNetV2 model for deployment by exporting it in a format compatible with your production environment, such as ONNX or TensorFlow SavedModel.

Benefits of fine-tuning

- + Cost savings: Fine-tuning is more affordable than developing a model from scratch. It reduces time, computing requirements, and the need for new hardware.
- + Improved accuracy for niche applications: Fine-tuning enhances performance for specialized tasks by refining a pre-trained model with a smaller, domain-specific data set.
- + Greater accessibility: Smaller organizations can leverage pre-trained models and fine-tune them with their own data, making advanced machine learning more attainable.

Challenges of fine-tuning

- + Due to the small data sets used in fine-tuning, you can risk overfitting your model. An overfitted model is so attuned to the training data that it doesn't perform well with new data.
- + Fine-tuning a model can risk having the pre-trained model "forget" some of its original training. This happens when you target specific layers too heavily or use a data set that's too different from the training set.
- + If the original pre-trained model has certain biases, flaws, or security issues, your fine-tuned model will also have those issues.

MobileNet

+

What is MobileNet?

- + MobileNet is a family of convolutional neural network (CNN) architectures designed for image classification, object detection, and other computer vision tasks.
- + They are designed for small size, low latency, and low power consumption, making them suitable for on-device inference and edge computing on resource-constrained devices like mobile phones and embedded systems.
- + They were originally designed to be run efficiently on mobile devices with TensorFlow Lite.

What is MobileNet?

- + The need for efficient deep learning models on mobile devices led researchers at Google to develop MobileNet.
- + As of 2025, the family has four versions, each improving upon the previous one in terms of performance and efficiency.
 - MobileNetV1 (17 April, 2017)
 - MobileNetV2 (21 March, 2019)
 - MobileNetV3 (20 November, 2019)
 - MobileNetV4 (29 September, 2024)

MobileNetV1

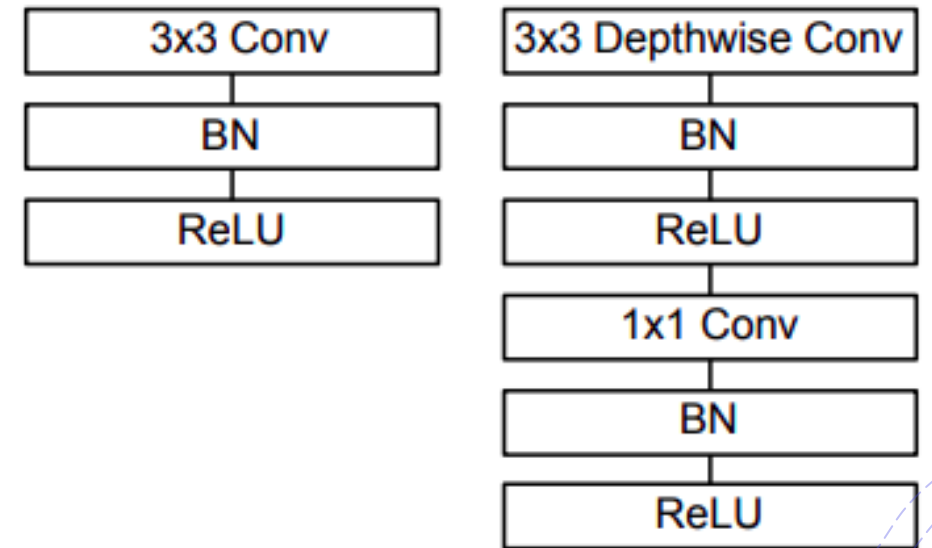
+

Architecture of MobileNetV1

- + MobileNetV1 was published in April 2017. Its main architectural innovation was incorporation of depth-wise separable convolutions.
- + Depth-Wise separable convolution originated from the idea that a filter's depth and spatial dimension can be separated, thus, the name separable.
- + The depth-wise separable convolution decomposes a single standard convolution into two convolutions: a depth-wise convolution that filters each input channel independently and a pointwise convolution (1×1 convolution) that combines the outputs of the depth-wise convolution. This factorization significantly reduces computational cost.

Architecture of MobileNetV1

+ For example, instead of using a single 3x3 convolution layer, MobileNet splits the convolution operation into a 3x3 depth-wise convolution and a 1x1 point-wise convolution.



Architecture of MobileNetV1

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
		$14 \times 14 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw
		$14 \times 14 \times 512$
	Conv / s1	$1 \times 1 \times 512 \times 1024$
		$7 \times 7 \times 512$
	Conv dw / s2	$3 \times 3 \times 1024$ dw
		$7 \times 7 \times 1024$
	Conv / s1	$1 \times 1 \times 1024 \times 1024$
		$7 \times 7 \times 1024$
	Avg Pool / s1	Pool 7×7
		$7 \times 7 \times 1024$
	FC / s1	1024×1000
		$1 \times 1 \times 1024$
	Softmax / s1	Classifier
		$1 \times 1 \times 1000$

MobileNetV2

+

Architecture of MobileNetV2

- + MobileNetV2 was published in March 2019. It uses **inverted residual layers** and **linear bottlenecks**.
- + **Inverted residuals** modify the traditional residual block structure. Instead of compressing the input channels before the depth-wise convolution, they expand them. This expansion is followed by a 1×1 depth-wise convolution and then a 1×1 projection layer that reduces the number of channels back down.
- + This inverted structure helps to maintain representational capacity by allowing the depth-wise convolution to operate on a higher-dimensional feature space, thus preserving more information flow during the convolutional process.

Architecture of MobileNetV2

+ **Linear bottlenecks** removes the typical ReLU activation function in the projection layers. This was rationalized by arguing that that nonlinear activation loses information in lower-dimensional spaces, which is problematic when the number of channels is already small.

Pros of MobileNet

- + Efficiency: MobileNet is designed specifically for mobile and embedded devices, making it highly efficient in terms of processing power and memory usage.
- + Speed: The model is optimized for fast inference, making it ideal for real-time applications such as object detection and image recognition on mobile devices.
- + Accuracy: Despite its focus on efficiency, MobileNet still achieves competitive accuracy in various computer vision tasks, making it suitable for a wide range of applications.

Cons of MobileNet

- + Complex tasks: MobileNet may not be as suitable for complex vision tasks that require a higher level of precision and detailed feature extraction.
- + Limited scalability: While optimized for mobile and embedded devices, MobileNet may not be as scalable or adaptable to larger, more powerful hardware systems.
- + Trade-off between speed and accuracy: In some cases, the speed optimization of MobileNet may come at the cost of some degree of accuracy, particularly in challenging vision scenarios.



Thank You